BACHELOR THESIS PLAYBOOK

Multimodal AI Workspace for Documents and Images

A practical roadmap for building a multimodal workspace that understands documents, screenshots, diagrams and images through natural-language interaction.

Goal: Build a polished application that lets users upload mixed visual and textual content, ask questions about it, extract structured information and compare sources using modern Vision-Language Models and Retrieval-Augmented Generation.

1. Project at a Glance

You will build a workspace where users upload PDFs, images, screenshots or diagrams and interact with them through natural language. The system should combine direct multimodal understanding with retrieval over extracted text so that answers remain grounded in the uploaded material. The emphasis is on a reliable application, not on training a model.

Learning outcomes

- Build a complete multimodal document workspace with upload, indexing, search and chat.
- Use a Vision-Language Model for screenshots, diagrams and image-heavy pages.
- Use Retrieval-Augmented Generation for longer textual documents.
- Generate validated structured outputs such as extracted fields, summaries or comparison tables.
- Evaluate accuracy, latency and cost across alternative multimodal pipelines.

2. Scope: What Must Be Built

MVP - required for a complete thesis

- User workspace with upload and document/image management.
- Support at least PDF, image and screenshot inputs.
- Text extraction and chunking for searchable documents.
- Natural-language Q&A grounded in the selected files.
- VLM-based interpretation of image-heavy content and diagrams.
- Structured extraction for at least one use case, such as requirements, entities or key fields.
- Comparison mode for two selected documents or images.

Stretch goals - only after the MVP works

- Cross-document contradiction detection.
- Automatic document classification and tagging.
- Visual citations or highlighted evidence regions.
- Local/private processing for selected files.

3. Suggested System Architecture

Component	Responsibility
Client application	Workspace UI for files, previews, chat, extraction and comparison.
Backend/API	File handling, metadata, orchestration and persistence.
Document pipeline	Text extraction, chunking, embeddings and vector retrieval.
Multimodal layer	VLM for screenshots, diagrams, images and visually complex pages.
Grounding layer	Combines retrieved text and multimodal observations before generation.

4. Data Strategy

No training dataset is required. Users provide files at runtime. For evaluation, create a small fixed benchmark of representative documents and screenshots with manually written expected answers or extracted fields. Use public or self-created materials so the experiment is repeatable and legally safe.

5. Modern AI Component

- Use a VLM for visual understanding and an LLM/RAG pipeline for long textual context.
- Keep extraction schemas explicit so outputs can be validated programmatically.
- Return evidence references to the source file or page whenever possible.
- Use a fallback strategy when OCR/text extraction is poor or the page is mainly visual.

6. Evaluation Plan

Your thesis should include at least one repeatable experiment. The goal is not to prove that AI is “good”, but to measure whether your design choices improve the system.

- Compare direct VLM understanding with OCR + LLM for selected tasks.
- Measure answer/extraction accuracy on a small manually labelled benchmark.
- Measure end-to-end latency and approximate model cost per task.
- Compare at least two models or pipelines on the same inputs.
- Optional experiment: RAG versus sending full extracted context directly to the model.

7. Risks and Mitigations

Risk	Mitigation
Too many file formats	Limit the MVP to PDF and common image formats.
Answers not grounded in files	Require retrieved evidence and reject unsupported claims where possible.
Large documents exceed context limits	Use chunking and retrieval instead of sending whole documents.
Evaluation becomes subjective	Use concrete extraction/Q&A tasks with expected answers.

8. Suggested 14-Week Execution Plan

Period	Target
Weeks 1-2	Problem definition, requirements, related work and technology spike.
Weeks 3-4	Core application architecture, database model and basic UI flow.
Weeks 5-6	Implement the non-AI core features and deterministic business logic.
Weeks 7-8	Integrate the LLM/VLM/agent component with structured outputs and error handling.
Weeks 9-10	Complete the end-to-end workflow, validation and user-facing editing.
Weeks 11-12	Evaluation experiments, metrics, fixed test scenarios and performance measurements.
Week 13	Polish, bug fixing, demo preparation and thesis figures/screenshots.
Week 14	Final evaluation, thesis consolidation and presentation rehearsal.

9. Suggested Thesis Structure

- 1. Introduction and motivation
- 2. Related work and enabling technologies
- 3. Requirements and system architecture
- 4. Design and implementation
- 5. AI integration and safeguards
- 6. Evaluation methodology and results
- 7. Discussion, limitations and future work
- 8. Conclusions

10. Final Deliverables

- A working application with a complete end-to-end user flow.
- Source code in a clean repository with setup instructions.
- A short architecture diagram and explanation of major design decisions.
- An evaluation script/notebook or reproducible test set with reported metrics.
- A thesis document that clearly distinguishes your engineering contribution from external AI services.
- A reliable live demo plus a recorded backup demo.

11. Supervisor Checkpoints

- Checkpoint 1: requirements, MVP and technology choices agreed.
- Checkpoint 2: core application works without the AI layer.
- Checkpoint 3: AI workflow works end-to-end with structured outputs and failures handled.
- Checkpoint 4: evaluation is defined before final polishing begins.
- Checkpoint 5: final demo is repeatable and the thesis contains measured results.

12. Concrete Demo Scenario

The user uploads a product specification PDF, a UI screenshot and an architecture diagram. The system summarizes the specification, answers a question with page-level evidence, explains the diagram, extracts key requirements into a structured list and compares the screenshot against the written requirements.

13. Recommended Technical Approach

- Choose a familiar client stack (for example React, Flutter, SwiftUI or another framework you can deliver reliably).
- Use a small backend service for persistence, external API access and AI orchestration.
- Prefer existing foundation-model APIs or local models instead of training a model.
- Use structured outputs, validation and deterministic business logic around the LLM/VLM/agents.
- Keep secrets and API keys on the server side and add basic logging for debugging.
- Write integration tests for the main workflow and unit tests for deterministic logic.

14. What NOT to Spend Time On

- Training a custom OCR or vision model.
- Supporting every office file format in the MVP.
- Building a full collaborative document editor.
- Relying on free-form model output when structured data is required.

Success criterion: A smaller product that works reliably, is evaluated properly and is easy to demonstrate is better than an ambitious system with unfinished features.